



Εθνικό και Καποδιστριακό Πανεπιστήμιο Αθηνών
Τμήμα Πληροφορικής και Τηλεπικοινωνιών

Ανάπτυξη Λογισμικού για Πληροφοριακά Συστήματα
Εργασία III

Ονοματεπώνυμο: Ιωάννης Γρυπιώτης

Αριθμός Μητρώου: 1115202100028

Ονοματεπώνυμο: Ιωάννης Κούβελας

Αριθμός Μητρώου: 1115202100070

Ονοματεπώνυμο: Πάντιος Ματιάτος

Αριθμός Μητρώου: 1115202100098

Περιεχόμενα

1	Εισαγωγή	2
2	Αλγόριθμοι	2
2.1	<i>GreedySearch</i>	2
2.2	<i>RobustPrune</i>	2
2.3	<i>Vamana</i>	2
2.4	<i>FilteredGreedySearch</i>	2
2.5	<i>FilteredRobustPrune</i>	2
2.6	<i>FilteredVamana</i>	3
2.7	<i>StitchedVamana</i>	3
3	Πειράματα με αλλαγές στις παραμέτρους	3
3.1	Αλλαγή του α	3
3.2	Αλλαγή του k	5
3.3	Αλλαγή του L, L_{small}	7
3.4	Αλλαγή του R, R_{small}	9
4	Βελτιστοποιήσεις	11
4.1	Γραφήματα Βελτιστοποίησης	11
4.2	<i>Distance Array</i>	12
4.3	<i>Updated Euclidean</i>	12
4.4	<i>Threads</i>	12
4.5	<i>Random Edges</i>	13
4.6	<i>Random Medoid</i>	13
4.7	<i>Final Merged</i>	14
5	Συμπέρασμα και Σχόλια	14

1 Εισαγωγή

Στην εποχή των μεγάλων δεδομένων, η ανάκτηση των πληροφοριών τους είναι πολύ σημαντική και κομβική. Εκεί εμφανίζονται οι αλγόριθμοι 'Αναζήτηση εγγύτερου γείτονα' που στοχεύουν στην εύρεση ενός σημείου σε ένα δεδομένο σύνολο. Στα πλαίσια αυτά υπάρχει το *DiskANN* το οποίο είναι ένα σύστημα που κατασκευάστηκε για να ανταπεξέλθετε ακόμη και σε αναζητήσεις γράφων τους ενός δισεκατομμυρίου. Αυτό επιτυγχάνεται με την χρήση ενός αλγορίθμου που λέγεται *Vamana* ο οποίος κατασκευάζει τον γράφο και με το *DiskANN* αποθηκεύουν αυτόν στον *SSD*, και παρά το μέγεθος ακόμη και του ένα δισεκατομμυρίου το σύστημα αυτό αποδίδει άριστα ακόμη και με μικρή μνήμη. Θα δούμε πώς η αλλαγή παραμέτρων όπως ο αριθμός γειτόνων (k), οι υποψήφιοι γείτονες (L), τον αριθμό των μέγιστων γειτόνων που μπορεί να έχει κάθε κόμβος (R) και άλλοι το επηρεάζουν και δίνουν διαφορετικά αποτελέσματα.

2 Αλγόριθμοι

2.1 *GreedySearch*

Ο αλγόριθμος *GreedySearch* μας παρέχει την δυνατότητα να προσεγγίσουμε ένα σημείο, ξεκινώντας από την αρχή κάποιου άλλου σημείου στον γράφο. Στην υπολοίψη του αλγορίθμου, χρησιμοποιήσαμε από την βιβλιοθήκη της *STL* την *std::set*, καθώς χρειαζόμαστε τα δεδομένα μας να είναι ταξινομημένα και το *set* μας παρέχει αυτόματη ταξινόμηση με την μικρότερη πολυπλοκότητα $O(\log n)$, αντίθετα με άλλα *data structures* όπως οι πίνακες και τα *vectors* που θα χρειαζόταν κάθε φορά να ξαναταξινομούμε με πολυπλοκότητα $O(n^2)$.

2.2 *RobustPrune*

Υπάρχουν ενδεχόμενα στα οποία οι γράφοι μπορεί να είναι πολύ μεγάλοι οπότε ο *GreedySearch* αλγόριθμος θα έβγαζε ένα 'μονοπάτι' αλλά θα ήταν αρκετά μεγαλύτερο γιατί θα επισκεπτόταν πολλούς από τους κόμβους του. Για να μετριάσει αυτό το πρόβλημα χρησιμοποιείται ο αλγόριθμος *RobustPrune* ο οποίος 'κλαδεύει' κόμβους από τον γράφο που ξεπερνούν την τιμή του μήκους επί του παράγοντα α . Όπως και στην *GreedySearch*, χρησιμοποιούμε *set* για τον ίδιο λόγο και χρησιμοποιούμε και πίνακες μέσα από το *struct* του γράφου, γιατί μας διευκολύνει στην εύρεση συγκεκριμένου στοιχείου, καθώς μπορούμε να το βρούμε με $O(1)$ πολυπλοκότητα αν έχουμε την θέση του.

2.3 *Vamana*

Χρησιμοποιώντας τους προηγούμενους αλγορίθμους *GreedySearch* και *RobustPrune* ο αλγόριθμος *Vamana* κατασκευάζει έναν κατευθυνόμενο γράφο. Παρόμοια και εδώ χρησιμοποιούμε τα ίδια *data structures* με πριν, συν τα *vectors* που τα χρησιμοποιούμε καθώς προσφέρουν δυναμική μνήμη και μπορούμε να βρούμε το σημείο που βρίσκεται κάποιο στοιχείο με πολυπλοκότητα $O(\log 1)$.

2.4 *FilteredGreedySearch*

Όπως και η *GreedySearch* έτσι και η *FilteredGreedySearch* κάνουν ακριβώς το ίδιο, με την διαφορά ότι στην *FilteredGreedySearch* χρησιμοποιούνται και φίλτρα. Ότι *data structure* αξιοποιούνται στην προηγούμενη συνάρτηση αξιοποιούνται και σε αυτήν.

2.5 *FilteredRobustPrune*

Παρομοίως η *FilteredRobustPrune* έχει την ίδια λογική με την *RobustPrune* απλά υπάρχει και χρήση φίλτρων. Επομένως και η *FilteredRobustPrune* έχει τα ίδια *data structures* με την *RobustPrune*.

2.6 FilteredVamana

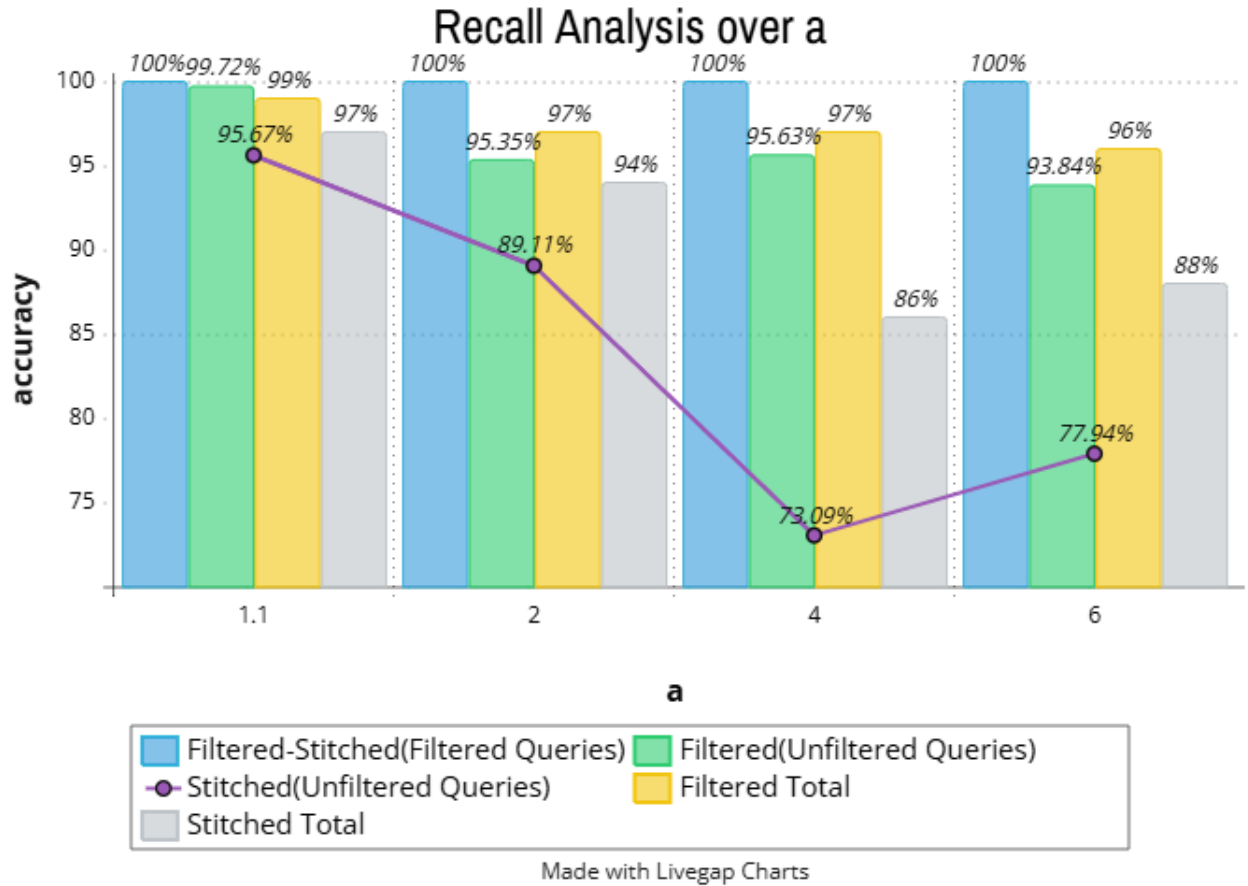
Με την ίδια ιδέα και η *FilteredVamana* είναι η *Vamana* με τις ίδες ακριβώς υλοποιήσεις και *data structures*, με την διαφορά ότι έχουμε τη χρήση φίλτρων.

2.7 StitchedVamana

Η *StitchedVamana* χρησιμοποιεί τις συναρτήσεις *GreedySearch*, *RobustPrune* και *Vamana*, οι οποίες έχουν διαμορφωθεί κατάλληλα για τα φίλτρα, για να κατασκευάσει πολλούς υπογράφους και με την βοήθεια της *FilteredRobustPrune* ενώνει όλους αυτούς του υπογράφους αφαιρώντας τις κοινές κορυφές. Ακόμη, σε αντίθεση με την *FilteredVamana* χρησιμοποιεί L και R που είναι μικρότερα για την γρηγορότερη κατασκευή του ευρετηρίου.

3 Πειράματα με αλλαγές στις παραμέτρους

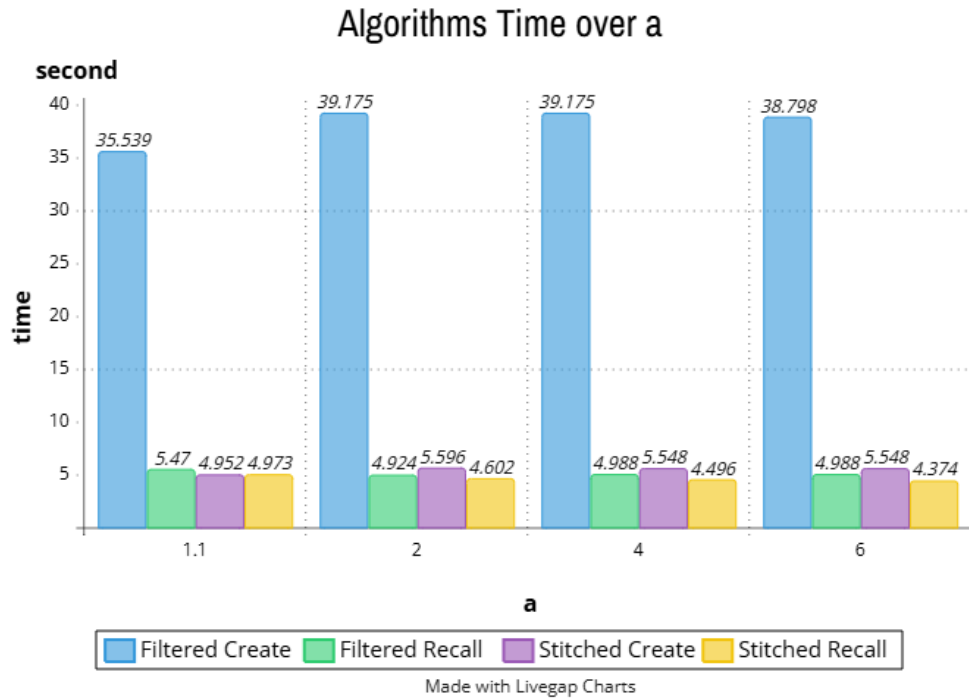
3.1 Αλλαγή του α



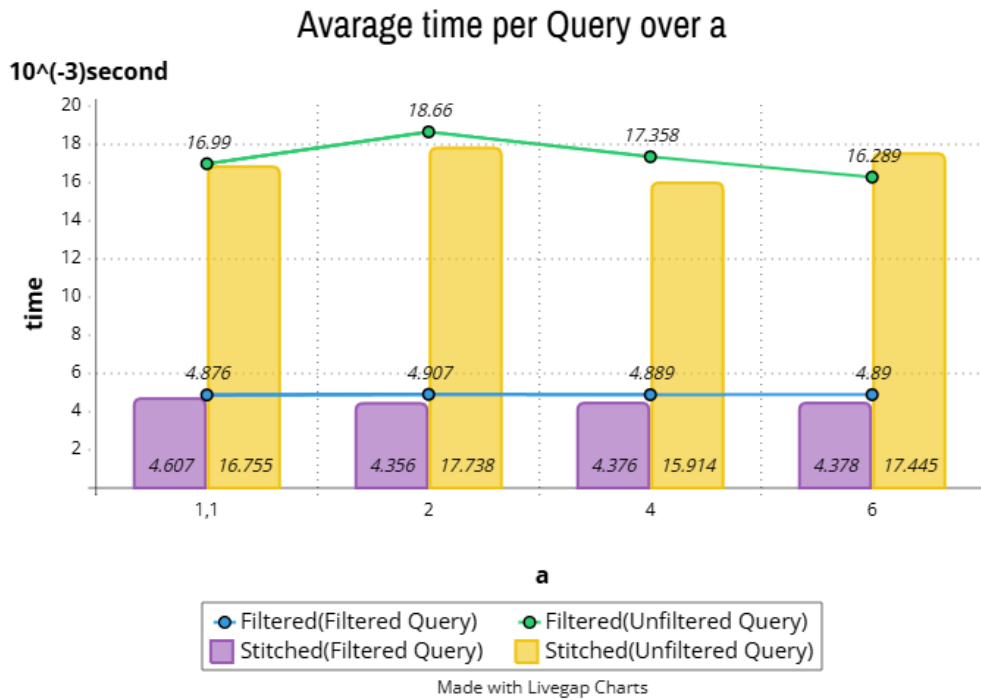
Στεθερές 1: " k " : 100, " L " : 200, " R " : 30, " L_{small} " : 100, " R_{small} " : 20, " $R_{stitched}$ " : 30

Παρατηρούμε στο πάνω γράφημα πώς όταν το α αυξάνεται από το 1.1 στο 2, 4 και 6 κάποια πράγματα αλλάζουν. Αν και τα φιλτραρισμένα *queries* παραμένουν σταθερά χωρίς καμία αλλαγή, τα άφιλτρα, είτε από την *FilteredVamana* είτε από την *StitchedVamana*, μειώνεται το ποσοστό επιτυχίας τους και μαζί τους και το συνολικό ποσοστό *accuracy* των *queries*. Ο λόγος που πέφτει το *accuracy* είναι επειδή το α είναι μεγαλύτερο

βλέπει περισσότερους γείτονες και δημιουργεί περισσότερα και μεγαλύτερα μονοπάτια και να ευθύνεται στις λιγότερες πιθανότητες να βρει τον κοντινότερο γείτονα.



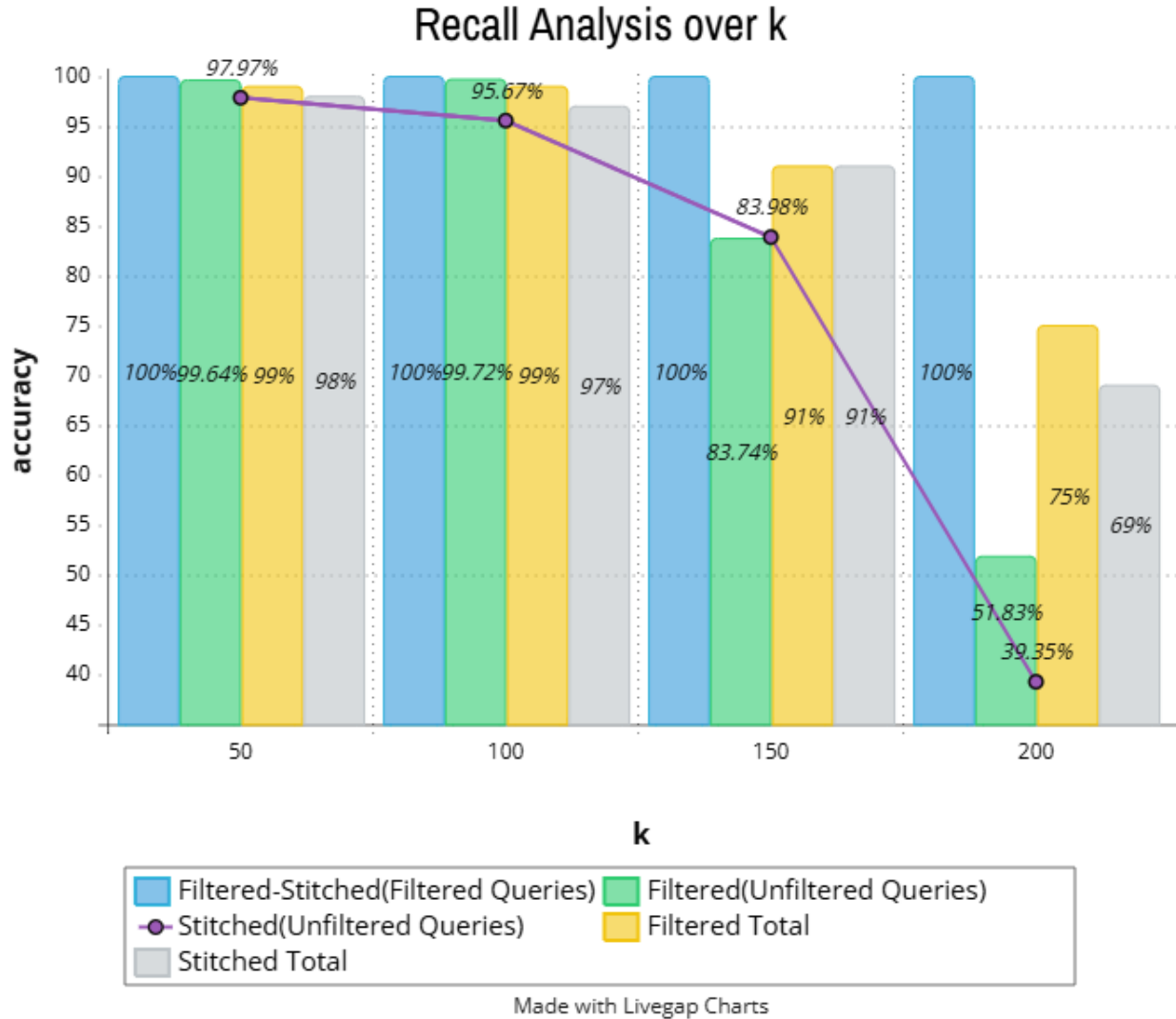
Στεθερές 2: "k" : 100, "L" : 200, "R" : 30, "L_{small}" : 100, "R_{small}" : 20, "R_{stitched}" : 30



Στα παραπάνω γραφήματα διακρίνουμε καθώς αυξάνεται το α οι αλλαγές στον χρόνο είναι πολύ μικρού μεγέθους. Εκτός από το χρόνο για να δημιουργηθεί η *Filtered* που φαίνεται η διαφορά λίγο περισσότερο με την

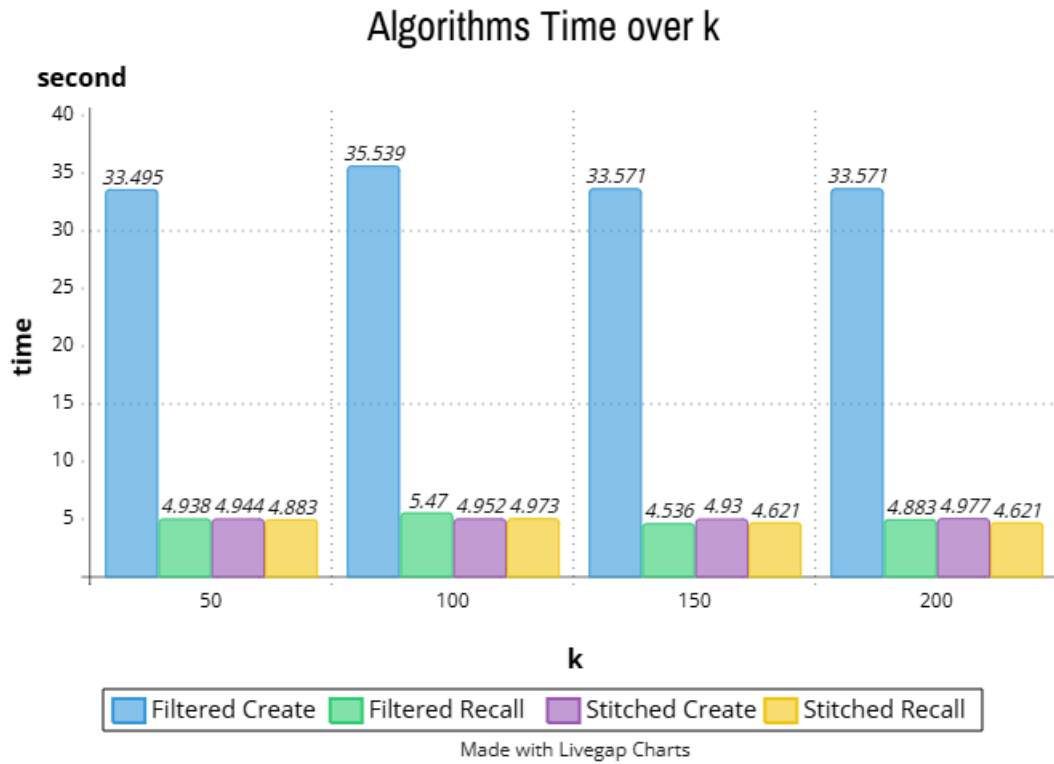
αύξηση να φτάνει μέχρι και τα 3 επιπλέον δευτερόλεπτα, οι υπόλοιποι χρόνοι παραμένουν σχεδόν σταθεροί, με την μεγαλύτερη διαφορά κοντά στα 0.6 δεύτερα. Αυτό συμβαίνει γιατί το α , αν και αυξήθηκε, δεν ήταν σε τόσο μεγάλο βαθμό για να επηρεάσει σημαντικά τους χρόνους.

3.2 Αλλαγή του k

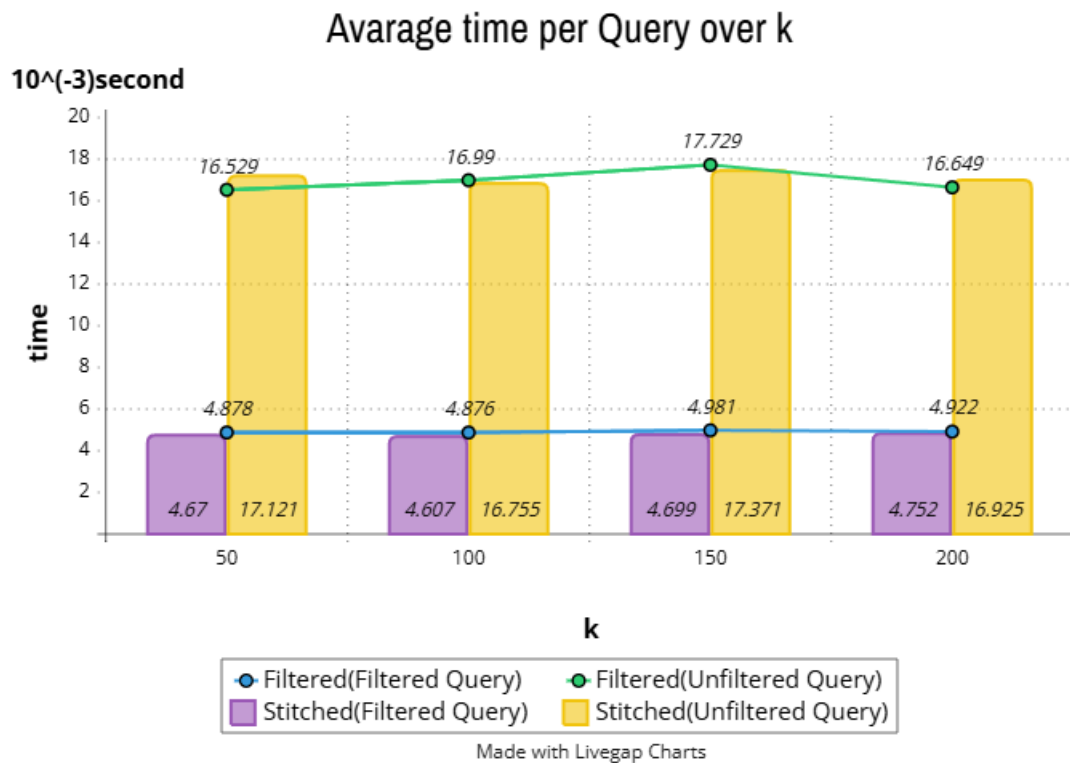


Στεθερές 3: " a " : 1.1, " L " : 200, " R " : 30, " L_{small} " : 100, " R_{small} " : 20, " $R_{stitched}$ " : 30

Στην αλλαγή του k , ενώ όλα τα άλλα παραμένουν σταθερά παρατηρούμε εμφανείς διαφορές στο γράφημα. Όταν το k αυξάνεται παρατηρούμε, αν και μεταξύ του 50 και 100 οι διαφορές είναι πάρα πολύ μικρές και σε κάποια σημεία στο $k = 100$ είναι καλύτερα σε απειροελάχιστο βαθμό, μετά το 100, το ποσοστό επιτυχίας για τα άφιλτρα, τα συνολικά φίλτρα και τα συνολικά φίλτρα της *Stitched* μειώνονται ραγδαία. Αυτό συμβαίνει γιατί όσο αυξάνεται το k και πηγαίνει πιο κοντά στην τιμή του $L(k \leq L)$, που L είναι οι πιθανοί γείτονες, αυξάνεται ο αριθμός των γειτόνων οπότε ο αλγόριθμος δεν βρίσκει πάντα τους σωτούς γείτονες αφού έχει ένα πολύ μεγάλο πλήθος από το οποίο θα πρέπει να επιλέξει.



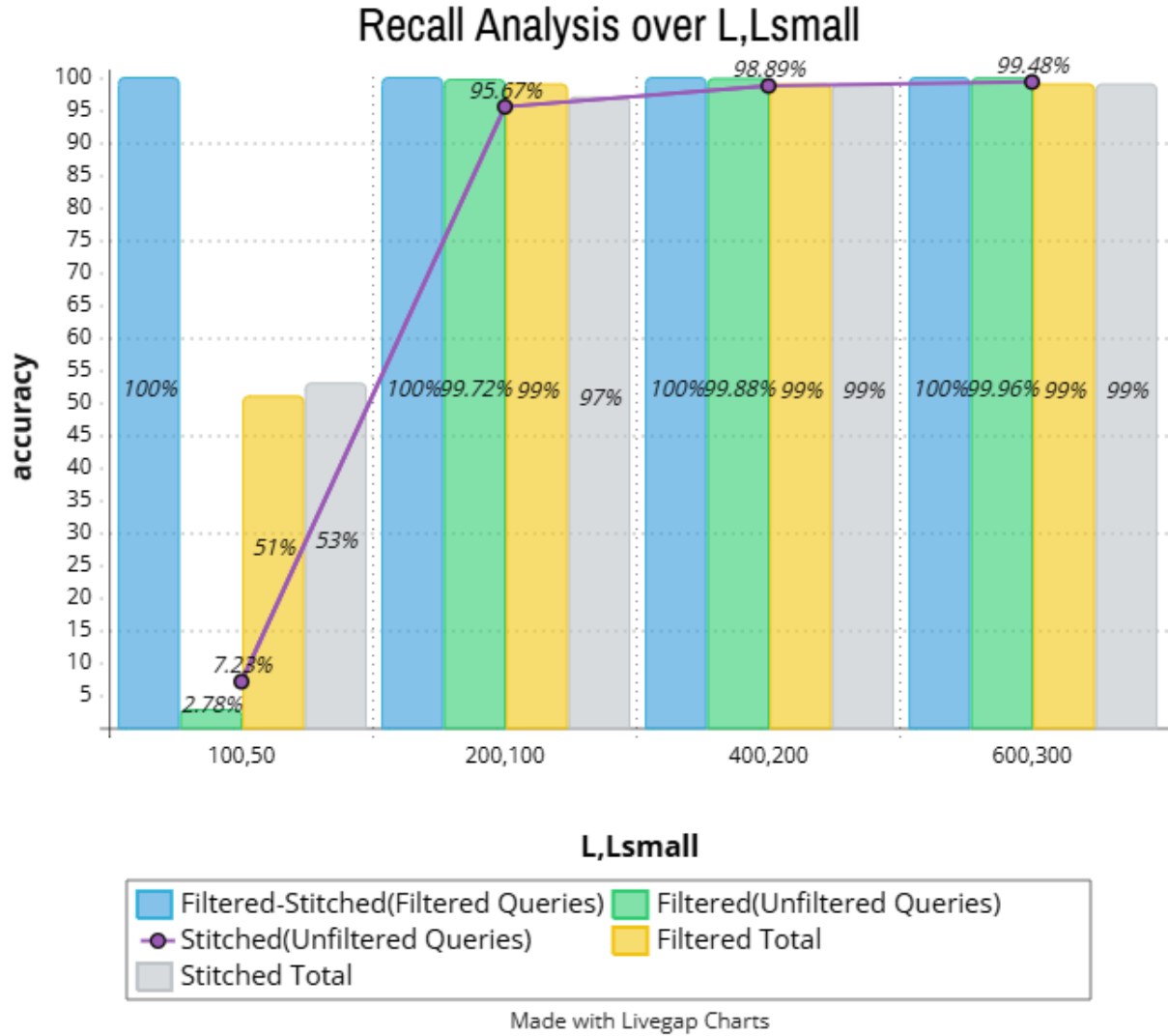
Στεθερές 4: "a" : 1.1, "L" : 200, "R" : 30, "L_{small}" : 100, "R_{small}" : 20, "R_{stitched}" : 30



Αντίστοιχα όταν έχουμε το k να μεγαλώνει οι χρόνοι παρατηρούμε ότι παραμένουν σχεδόν σταθερο-

ί με μικρές διαφορές. Αυτό είναι λογικό να συμβαίνει γιατί εκτός από το τέλος που βρίσκει ο αλγόριθμος *GreedySearch* του k κοντινότερους γείτονες, στην υπόλοιπη κατασκευή του γράφου και την εκτέλεση της *Filtered* και *Stitched* δεν παίζει κάποιο ρόλο.

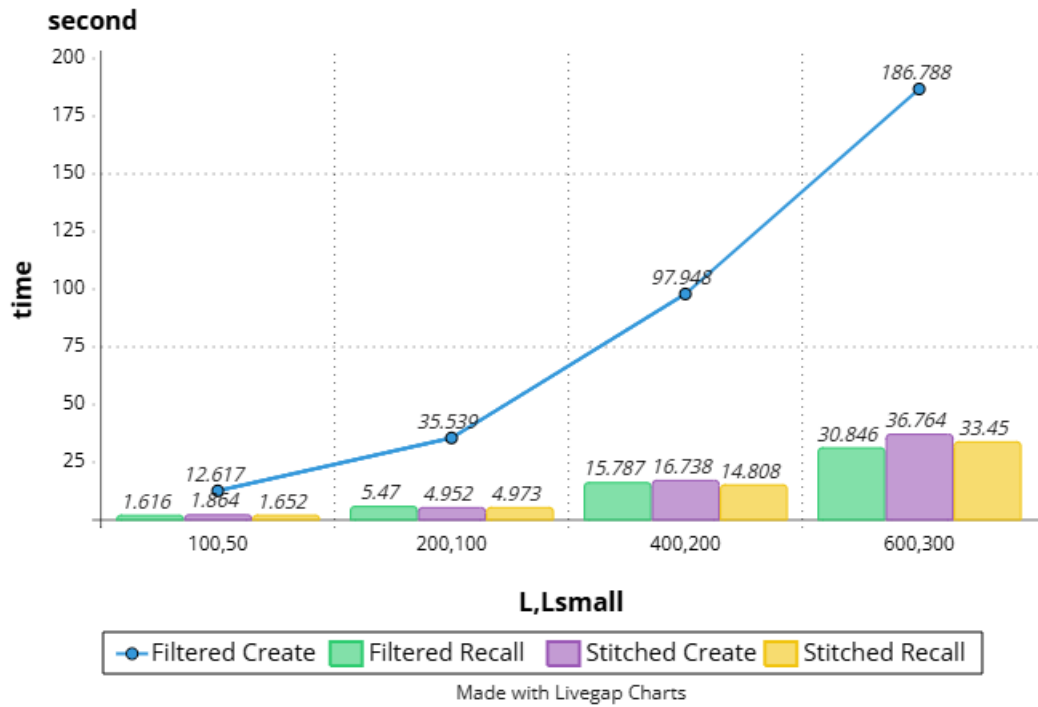
3.3 Αλλαγή του L, L_{small}



Στεθερές 5: " k " : 100, " a " : 1.1, " R " : 30, " R_{small} " : 20, " $R_{stitched}$ " : 30

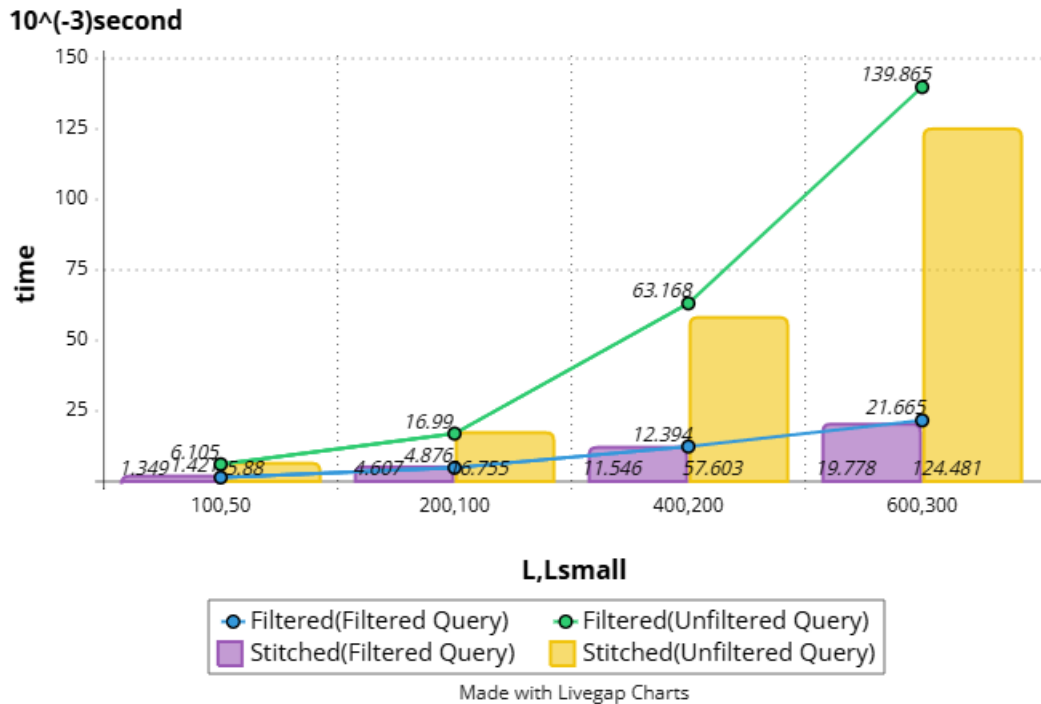
Στο διάγραμμα αυτό θα αλλάξουμε ταυτόχρονα το L για την *FilteredVamana* και L_{small} για την *StitchedVamana*, ενώ οι υπόλοιποι παραμέτρους παραμένουν σταθεροί. Όταν τα L, L_{small} είναι πολύ μικρά και το L έχει παρόμοια τιμή με το k , ή και στην συγκεκριμένη περίπτωση την ίδια, το *accuracy* θα είναι πάρα πολύ μικρό για το ίδιο λόγω που αναφέραμε και στην αλλαγή του k πιο πάνω, διότι το L που δείχνει τους πιθανούς γείτονες και το k τους κοντινότερους δυσκολεύει την *GreedySearch* να βρει τους σωστούς κοντινότερους γείτονες και τους εμφανίζει όλους οπότε και το ποσοστό επιτυχίας πέφτει. Ωστόσο, όταν το L αυξάνεται, και αντίστοιχα και το L_{small} , και η τιμή του γίνεται μεγαλύτερη από το k , το ποσοστό επιτυχίας αυξάνεται πολύ εφόσον πλέον ο αλγόριθμος μπορεί να βρει πιο εύκολα τους κοντινότερους γείτονες αφού έχει και μεγαλύτερο πλήθος από αυτών.

Algorithms Time over L,Lsmall



Στεθερές 6: "k" : 100, "a" : 1.1, "R" : 30, "R_{small}" : 20, "R_{stitched}" : 30

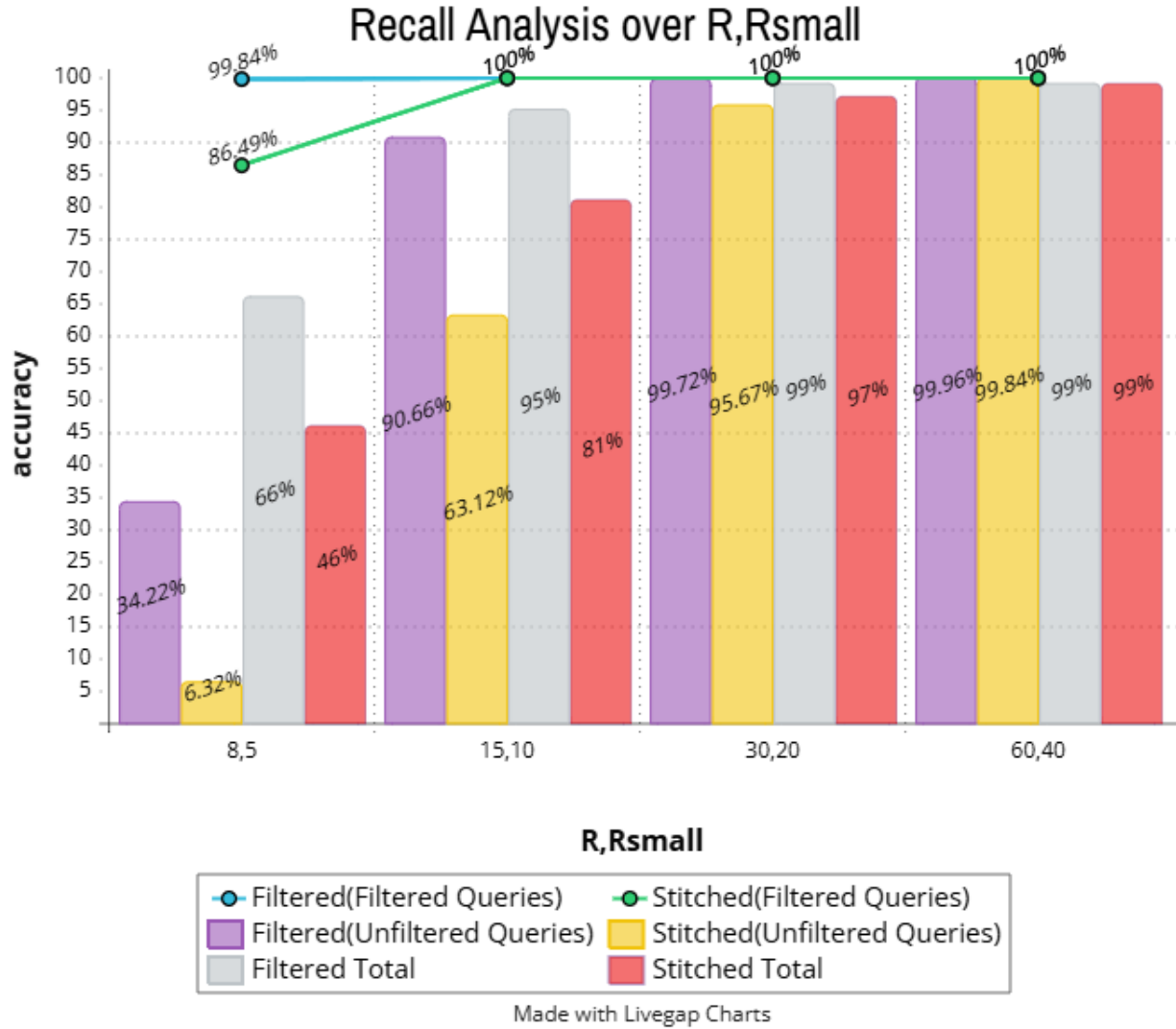
Avarage time per Query over L,Lsmall



Η αλλαγή των L, L_{small} θα δούμε πως έχει μεγάλη επιρροή στον χρόνο κατασκευής της *Filtered* και

Stitched. Όταν αυτά αυξάνονται και καθώς διασχίζουμε τον γράφο στην προσπάθεια του αλγορίθμου να βρεί το συντομότερο δρόμο για κάποιο κόμβο, πρέπει να διατρέξει τους γείτονες για να βρει ποιος είναι ο πιο κοντινός κάθε φορά, και επομένως όσο μεγαλύτερο είναι το L , L_{small} , τόσοι περισσότεροι γείτονες θα πρέπει να ελέγξει και αίτιο έχει ο χρόνος να αυξάνεται.

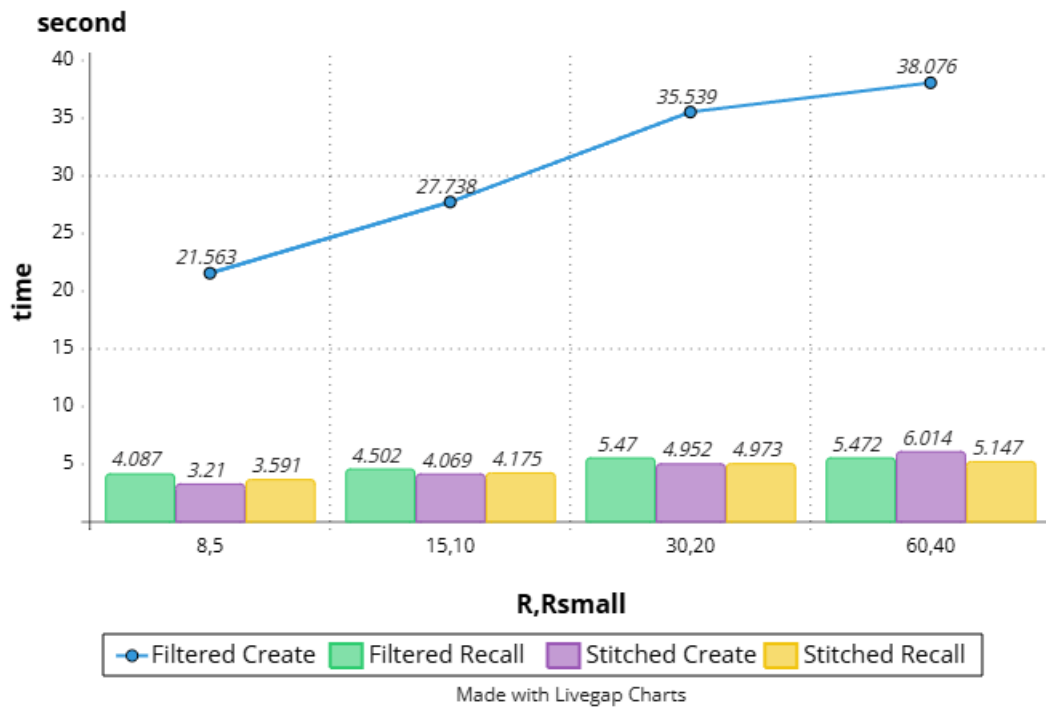
3.4 Αλλαγή του R , R_{small}



Στεθερές 7: " k " : 100, " a " : 1.1, " L " : 200, " L_{small} " : 100, " $R_{stitched}$ " : 30

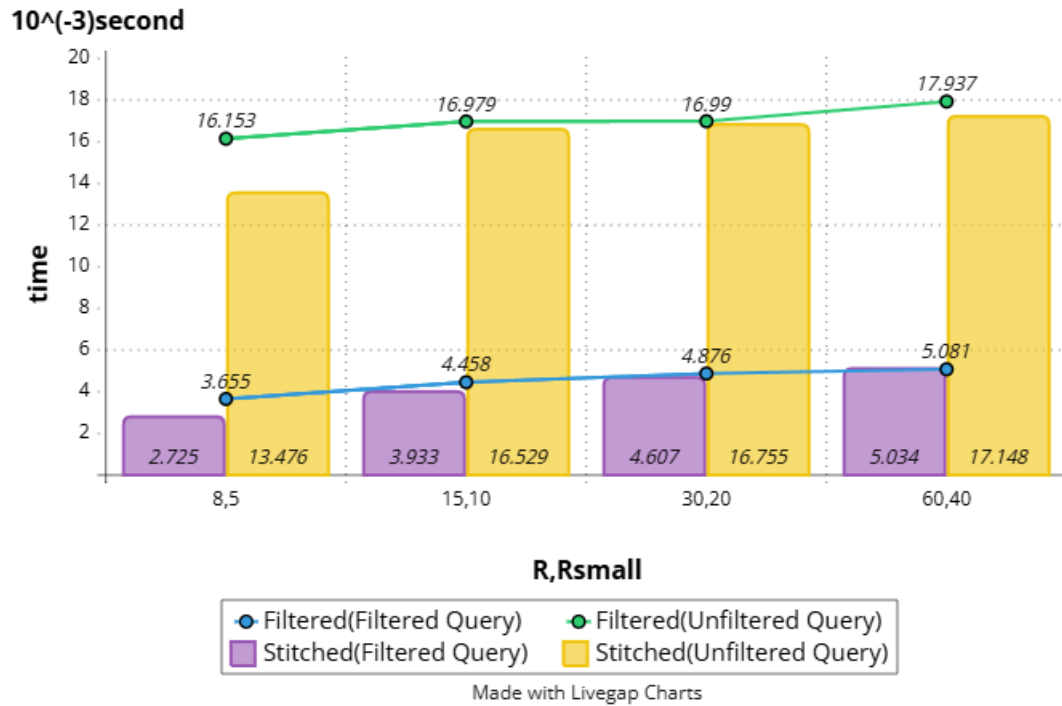
Το R , R_{small} μας δείχνει τον αριθμό των εξερχόμενων κόμβων από κάποιον κόμβο. Όταν μεγαλώνει ο αριθμός αυτός προσφέρει μεγαλύτερο ποσοστό επιτυχίας καθώς ο κάθε κόμβος έχει περισσότερους εξερχόμενους γείτονες, οπότε και ο γράφος είναι καλύτερα συνδεδεμένος προσφέροντας περισσότερους τρόπους για να διασχίσει τον γράφο στην αναζήτησή του για κάποιο συγκεκριμένο *query*.

Algorithms Time over R,Rsmall



Στερεός 8: "k" : 100, "a" : 1.1, "L" : 200, "L_{small}" : 100, "R_{stitched}" : 30

Avarage time per Query over R,Rsmall

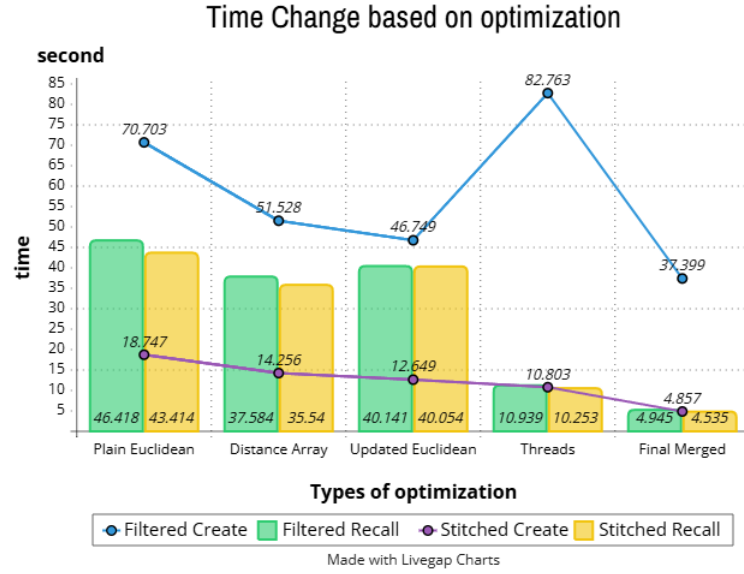


Ταυτόχρονα, είδαμε ότι όταν αυξάνεται το R, R_{small} το *accuracy* αυξάνεται και αυτό λόγω των περισ-

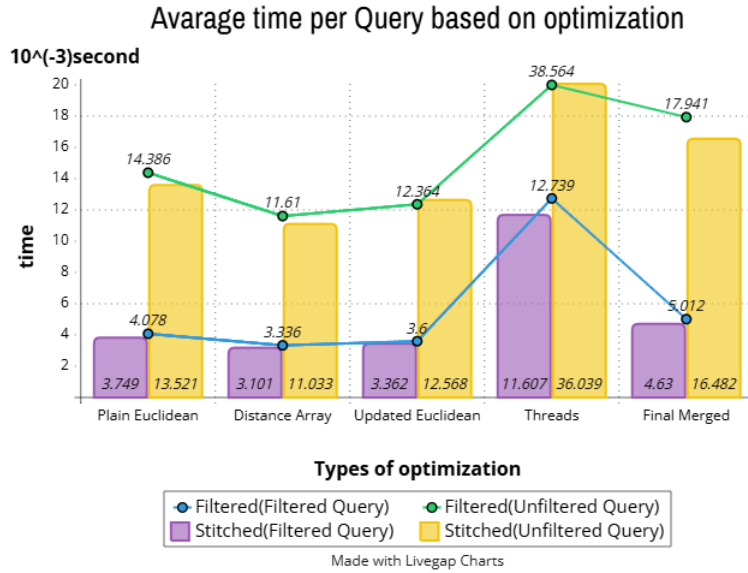
σότερων γειτόνων και την συνδεσιμότητα του γράφου. Παράλληλα αυξάνεται και ο χρόνος κατασκευής των αλγορίθμων επειδή θα πρέπει για κάθε κόμβο να δημιουργήσει περισσότερα μονοπάτια μεταξύ όλων των κόμβων, και γενικά επειδή αυξάνεται η συνδεσιμότητα επειδή μπαίνουν περισσότερα *edges*, ο χρόνος είναι λογικό να αυξηθεί.

4 Βελτιστοποιήσεις

4.1 Γραφήματα Βελτιστοποίησης



Στεθερές 9: "*k*" : 100, "*L*" : 200, "*R*" : 64, "*a*" : 1.1, "*L_ssmall*" : 100, "*R_ssmall*" : 20, "*R_sstitched*" : 64



Στεθερές 10: "*k*" : 100, "*L*" : 200, "*R*" : 64, "*a*" : 1.1, "*L_ssmall*" : 100, "*R_ssmall*" : 20, "*R_sstitched*" : 64

4.2 Distance Array

Η παραπάνω βελτιστοποίηση μειώνει το χρόνο εκτέλεσης τόσο κατά την δημιουργία των γράφων στον *FilteredVamana* και *StitchedVamana* όσο και στην εξακρίβωση του ποσοστού επιτυχίας των παραπάνω αλγορίθμων. Κατά την εκτέλεση των αλγορίθμων απαιτείται ο υπολογισμός κάποιων αποστάσεων ο οποίος είναι πολύ χρονοβόρος για το σύστημα. Αποθηκεύοντας τις αποστάσεις σε έναν πίνακα εάν μία απόσταση έχει ήδη υπολογιστεί ο αλγόριθμος την ανακτάει από τον πίνακα μειώνοντας δραματικά τον χρόνο (*time complexity for this distance* $O(1)$ instead of $O(n * d)$ where n the number of coordinates for the vector (or query) and d the dimension). Μπορούμε αφενώς να συμπεράνουμε ότι αν πολλές αποστάσεις εμφανίζονται επανειλημμένα για ένα συγκεκριμένο *query* τότε ο χρόνος με την παραπάνω βελτιστοποίηση μειώνεται εξίσου πολύ.

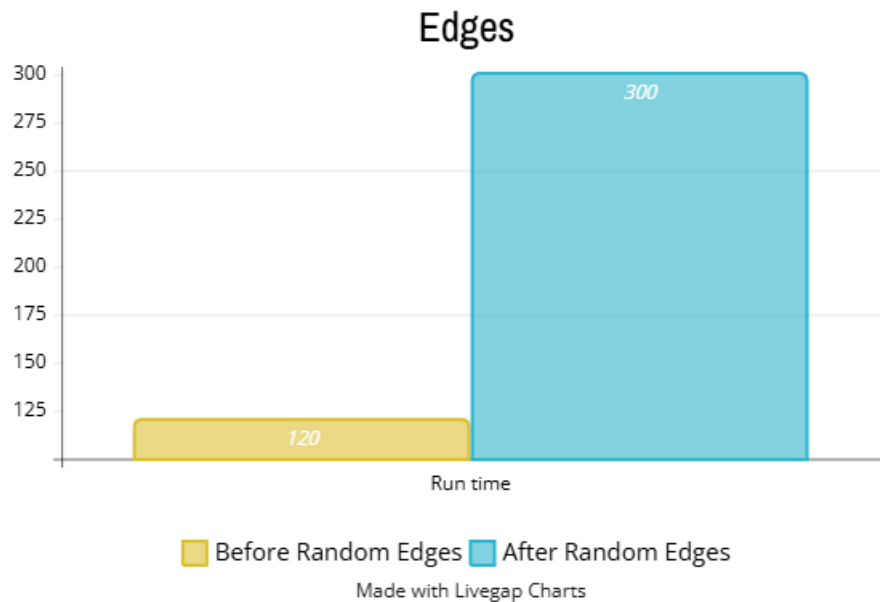
4.3 Updated Euclidean

- **Εκτέλεση πολλαπλών υπολογισμών ταυτόχρονα** Με τις εντολές *AVX*, επεξεργάζονται 8 στοιχεία ταυτόχρονα (για δεδομένα τύπου *float*), σε αντίθεση με τις παραδοσιακές εντολές που επεξεργάζονται κάθε στοιχείο ξεχωριστά. Έτσι, ο υπολογισμός των διαφορών, των τετραγώνων και της συσσώρευσης πραγματοποιείται πολύ πιο γρήγορα.
- **Μείωση επαναλήψεων** Χρησιμοποιώντας *AVX*, ο αριθμός των επαναλήψεων μειώνεται σημαντικά. Αντί να υπολογίζουμε κάθε στοιχείο σε ξεχωριστό βήμα, επεξεργαζόμαστε 8 στοιχεία κάθε φορά. Για παράδειγμα, σε έναν πίνακα 800 στοιχείων, χρειαζόμαστε 100 επαναλήψεις αντί για 800.
- **Αποδοτική χρήση καταχωρητών** Οι *AVX* εντολές χρησιμοποιούν εξειδικευμένους καταχωρητές πλάτους 256-bit, οι οποίοι είναι σχεδιασμένοι για να αποθηκεύουν και να επεξεργάζονται ταυτόχρονα πολλαπλά στοιχεία. Αυτό μειώνει τη συχνή πρόσβαση στη μνήμη, η οποία είναι αργότερη συγκριτικά.

4.4 Threads

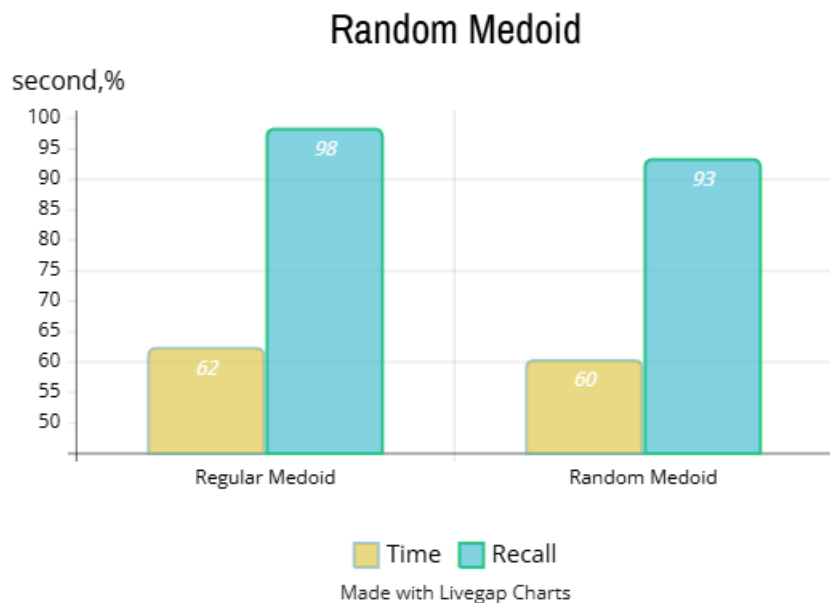
Σε ότι αφορά την εξακρίβωση του ποσοστού επιτυχίας η παράλληλη επεξεργασία με την χρήση νημάτων έχει μειώσει πολύ τον χρόνο εκτέλεσης σε σχέση με τις αρχικές μας υλοποιήσεις. Συγκεκριμένα η εύρεση του ποσοστού επιτυχίας εκτελείται ταυτόχρονα για ένα σύνολο πολλών *queries*, το καθένα σε διαφορετικό νήμα. Ομοίως η χρήση νημάτων εξοικονομεί πολύ χρόνο κατά την δημιουργία του γράφου στον αλγόριθμο *Stitched* εφόσον κάθε δημιουργία υπογράφου (όπου κάθε υπογράφος αντιστοιχεί σε ένα συγκεκριμένο φίλτρο) εκτελείται ταυτόχρονα/παράλληλα σε διαφορετικά νήματα. Ωστόσο η χρήση νημάτων στον αλγόριθμο *FilteredVamana* δεν απορρέει στην μείωση του χρόνου εκτέλεσης. Επειδή τα νήματα αποκτούν πρόσβαση σε κοινά δεδομένα απαιτείται η χρήση *std::lock_guard* ενός μηχανισμού συγχρονισμού για την αποφυγή προβλημάτων *data races*. Επειδή οι κλειδώσεις είναι συχνές και συνήθως μεγάλης διάρκειας ο συνολικός χρόνος έχει ως αποτέλεσμα να αυξάνεται αντί να μειώνεται σε σχέση με τις αρχικές μας υλοποιήσεις. Για αυτόν τον λόγο στο τελικό παραδοτέο ο αλγόριθμος *FilteredVamana* δεν χρησιμοποιεί νήματα.

4.5 Random Edges



Μία από τις βελτιστοποιήσεις που υπάρχει είναι η χρήση τυχαίων ακμών (όπως αναφέρεται και στο χαρτί) για την δημιουργία των γράφων με την χρήση της *FilteredVamana* και *StitchVamana*. Στην πραγματικότητα και στην υλοποίηση όμως δεν καταφέρνει κάτι καλύτερο αλλά αντ'αυτού ο χρόνος για την κατασκευή αυξάνεται κατά πολύ. Αποτέλεσμα αυτού είναι ότι η *FilteredVamana* επιλέγει ακμές με τα φίλτρα και όταν γίνεται τυχαία είναι πολύ πιθανό και όπως συμβαίνει ο χρόνος να αυξάνεται. Αντίστοιχα και με την *StitchVamana* που αξιοποιεί τυχαία αρχικοποίηση γειτόνων, δεν γίνεται πάντα να είναι αποτελεσματική. Η τυχαία προσέγγιση μπορεί να λειτουργεί σε ορισμένες περιπτώσεις, αλλά η εμπειρία μας δείχνει ότι η Ελεγχόμενη Βελτίωση της απόδοσης υποδεικνύει ότι η συγκεκριμένη μέθοδος ίσως δεν είναι κατάλληλη για όλες τις περιστάσεις.

4.6 Random Medoid



Η χρήση της *Random Medoid* από την κανονική έχει κάποια θετικά και αρνητικά. Διακρίνουμε πως όταν χρησιμοποιούμε την συγκεκριμένη *medoid* ο χρόνος μειώνεται και αυτό γιατί η *medoid* είναι από τις πιο χρονοβόρες διαδικασίες του συστήματος. Ταυτόχρονα, το ποσοστό επιτυχίας μειώνεται καθώς δεν είναι η καλύτερη μέθοδος αλλά επειδή παραμένει άνω των 90%, δηλαδή αυτό σημαίνει ότι ο αλγόριθμος παραμένει αποδοτικός είναι ένα ικανοποιητικό αποτέλεσμα.

4.7 *Final Merged*

Στο *Final Merged* χρησιμοποιήσαμε όλες τι από πάνω βελτιστοποιήσεις, αλλά κυρίως βελτιστοποιήσεις οι οποίες επέφεραν σοβαρές αλλαγές στον κώδικα.

5 Συμπέρασμα και Σχόλια

Όλα τα πειράματα και τις βελτιστοποιήσεις του κώδικα τις τρέξαμε σε ένα υπολογιστή με αυτό το *hardware*:

- *CPU : AMD Ryzen 5 7600X 4.7 GHz 6-Core Processor*
- *GPU : RTX 4060 8GB GDDR6*
- *RAM : G.Skill Flare X5 32 GB (2 x 16 GB) DDR5-6000 CL36 Memory*

Επιπλέον, προκειμένου να φτιάξουμε όλα τα γραφήματα τρέξαμε το πρόγραμμά μας αρκετές φορές προκειμένου να πάρουμε έναν μέσο όρο για να είναι όσο τον δυνατό πιο αντικειμενικά τα αποτελέσματα. Αν τρέχαμε τον κώδικα μόνο μια φορά τα αποτελέσματα μπορεί να μην ήταν τόσο αντικειμενικά μιας και θα είχαμε κάποιες μικροαλλαγές λόγω της φύσης των αλγορίθμων που είχαμε να υλοποιήσουμε.